

Mock Checkout

A buyer buys a digital product. The server prices it, a provider is charged, and a *signed provider event* — never the browser — is what confirms the payment. The order domain and the payment state machine, complete; the ledger, deliberately untouched.

Branch `claude/create-app-repository-1cxsih`Head `e9e16d5`

1 commit • 30 files

Tests **284 passed** • 22 browserMutations **11 of 11 caught**

1

Payment confirmed is not financial settlement

THE LINE THIS MILESTONE STOPS AT

Payment confirmed — a provider event, verified against the raw bytes we received and cross-checked against our own order, says a charge succeeded. That moves the payment, moves the order, freezes the commission split, and grants the buyer's entitlement.

Financial settlement — ledger postings that turn a captured payment into a seller's claim on money. **Not performed.** Not by this code, and deliberately not by a mock.

Why nothing is posted to the ledger

The obvious implementation would post the capture leg — `external_customer → psp_clearing` — the moment a payment confirms. I did not, for three reasons, and a test asserts the absence rather than leaving it to a comment:

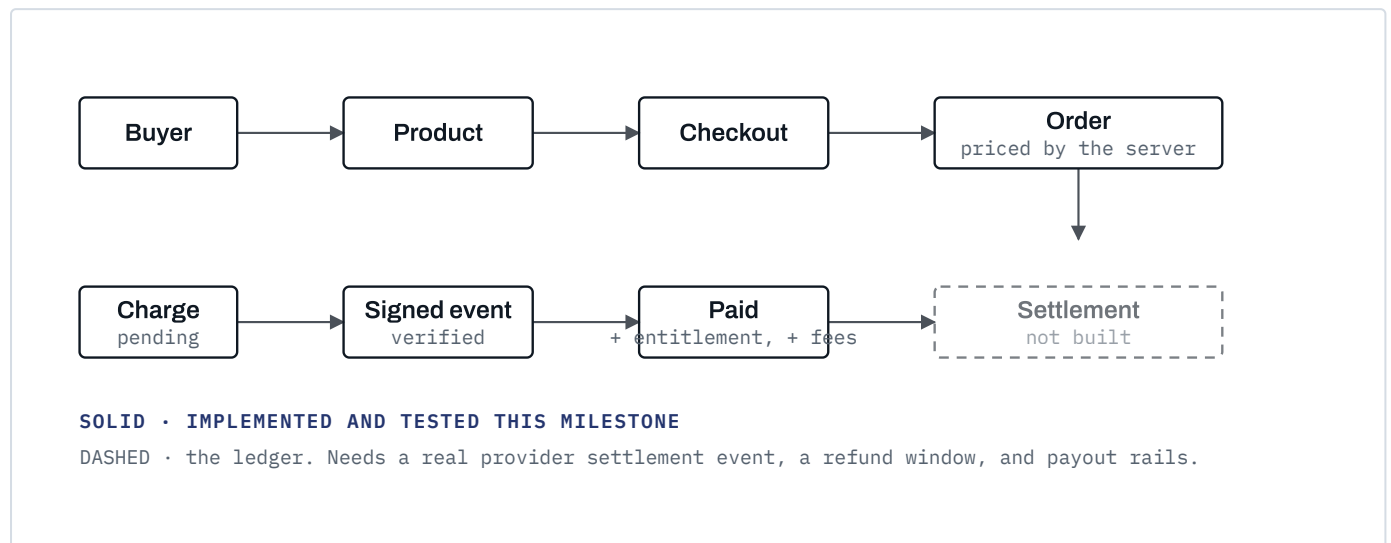
- **The ledger is append-only.** A posting that says "a customer paid 15 000 XOF" when a mock invented the payment is a false entry in a book that cannot be corrected by deletion — only by a compensating transaction that also has to be explained. The browser suite alone would write a dozen of them.
- **A capture is a claim about the outside world.** Afrinext does not observe it; a provider asserts it. Recording the assertion as a movement of value conflates "iPayMoney told us" with "the money exists", and those separate the moment a provider has an incident.
- **A seller entitlement before the funds arrive is a promise the balance cannot keep.** Splitting the gross into seller and platform shares creates a claim on money Afrinext does not yet hold, in a system whose whole point is that `account_balances` is a cache of something true.

What confirmation *does* write

The **frozen fee schedule**. `fee_schedules` and `fee_lines` were built in Phase 1 with a comment saying `subject_type` would be `'settlement'` now and `'order'` once orders existed; orders now exist, so it is `'order'`. The rows are append-only and must total the gross, so the sale's economics — 15 000 XOF gross,

2 700 platform, 12 300 seller — are permanent from the moment of confirmation. Change the commission rule next quarter and this order still says what it said today.

It moves no money. It records what the sale charged, so that settlement — whenever it is built — reads a frozen decision instead of re-resolving rules that have since changed.



The journey, and where it stops. Everything left of the dashed box is implemented; **the dashed box is the whole of settlement** and is not in this milestone. A test asserts `ledger_entries` is empty after a confirmed sale, so the boundary is enforced rather than described.

The state machines, declared once and enforced twice

Both machines live in `orders/state.ts` as data. Not as `if` chains at the call sites: a transition rule expressed in three places is a rule that will eventually disagree with itself.

ORDER	MAY BECOME	MEANING
pending_payment	paid · failed · cancelled · expired	Created and priced. The only non-terminal state.
paid	—	A provider confirmed a capture. Not "the seller has been paid".
failed	—	Terminal on purpose: a new attempt is a new checkout, with a fresh price read and a fresh expiry. Reusing an order across attempts is how a stale price gets charged.
cancelled · expired	—	Closed without payment.

PAYMENT	MAY BECOME	MEANING
initiated	pending · failed · cancelled · expired	The row exists; the provider has not answered.
pending	succeeded · failed · cancelled · expired	A charge is in flight. The order stays <code>pending_payment</code> .
succeeded · failed · cancelled · expired	—	Terminal. A later event claiming otherwise is recorded and ignored.

ENFORCED TWICE, AND THE SECOND ONE IS THE REAL ONE

Every mutation is a **guarded UPDATE** naming the state it expects — `where id = ... and status = 'pending_payment'`. A row count of zero is the rejection. So a caller that skipped the table in TypeScript still cannot write an impossible history, and two confirmations racing each other produce one winner and one no-op rather than two sets of effects.

One divergence that is deliberate

A payment can succeed while its order refuses to move — the checkout expired while the buyer was on the provider's page. The payment transitions, because it did: money may genuinely have left an account. The order stays `expired` and **no entitlement is granted**. That divergence is a real operational event and it is visible precisely because the two are separate rows with separate states. It is listed as an open risk in section 8, because nothing yet reconciles it.

3 The trust boundary: five checks before anything is believed

The browser is never believed. A buyer returning from a hosted checkout page with `?status=success` in the URL has proved only that they can type a URL. What is believed is a signed event, and only after five checks in this order:

1 The signature verifies over the RAW bytes

The body is handed to the provider unparsed. Parsing first and verifying the parsed shape is how signature checks get bypassed — the signature covers the bytes, not our interpretation of them. A test signs a valid event, alters one digit of the amount, and requires the same signature to be refused.

2 The provider's event id has not been seen

Unique index on `(provider, provider_event_id)`.

3 The reference resolves to a payment we created

Not to an order id, which a caller could guess. An event for an unknown charge is recorded as rejected and refused.

4 The amount and currency match exactly

Minor units compared as `bigint`, currency compared as a string. A near miss is a mismatch. 4 999 against a 5 000 order is refused, and so is 5 000 EUR against 5 000 XOF.

5 The transition is legal from the state the payment is actually in

Not from the state the event assumes.

THE ENDPOINT REQUIRES NO SESSION — AND THAT IS THE POINT

A provider has no cookie. Authentication at this endpoint is the signature. A browser test proves the consequence directly: the signed-in buyer posts a perfectly formed success event from inside their own session, and it is **refused**, because a session is not a signature and this handler has never cared about cookies.

Every refusal answers the same `400` and never says which check failed. Someone probing with forged amounts learns nothing from the response; the reason goes to `payment_events` and the audit log, where only we can read it.

What the client is allowed to say

A checkout request carries a store slug, a product slug, a quantity and an idempotency key. There is **no price, currency, total, seller or commission field** — not "validated and ignored", but absent, so there is

nothing for a future edit to start trusting.

A browser test sends `priceMinor: "1"`, `totalMinor: "1"`, `currency: "EUR"` and `commissionBps: 0` anyway. The order comes back at **9 000 XOF**.

Visibility is in the `WHERE` clause, not applied after the read: `p.status = 'published'` and `s.status = 'published'`, the same condition the public product page uses. A buyer cannot check out something they could not have seen, and a draft product is not fetched-then-refused — it is not found.

REPEATED THING	WHAT STOPS IT	PROVED BY
A buyer double-taps <i>Buy</i>	Unique <code>(buyer_user_id, checkout_key)</code> ; the insert is <code>on conflict do nothing</code> and the row is read back, so the index arbitrates rather than a read-then-write race	three concurrent submissions → one order, one line
A retried <i>Pay</i>	A partial unique index allows one <code>initiated / pending</code> payment per order; the provider is called with a derived key	the second call returns the first payment, one row
A provider retries a webhook	The guarded transition, then the unique event id	delivered three times → paid once, one grant, one fee schedule, one event row
Two deliveries arrive at once	Row locks plus the guarded <code>UPDATE</code>	five concurrent deliveries → exactly one applies
Two <i>different</i> events race for one charge	Same guard: the loser's <code>UPDATE</code> matches nothing	the order has one outcome, and the entitlement matches it
Two orders for one product are both confirmed	Unique <code>(user_id, product_id)</code> on entitlements	both orders paid, one grant — access is not a quantity

WHY ONE TRANSACTION, AND WHY THE EVENT ROW IS WRITTEN LAST

Everything a confirmation changes happens in a single transaction: the guarded payment update, the guarded order update, the entitlement, the frozen fees, and finally the event record. The guard comes first because it is what makes the whole thing idempotent. The event row comes last because it then carries *what actually happened* rather than what we hoped would happen when we started.

Committing the status and repairing the rest afterwards was the first design, and it is wrong: a missing commission rule would leave a paid order with no record of what it charged, and the provider's retry would refuse to touch it because the event id had been seen. Rolling everything back leaves the provider a clean slate to retry into.

The provider port, and what the mock does not prove

STATED PLAINLY

No iPayMoney integration was added. `IPayMoneyProvider` still throws on every method, and its documentation is still unavailable. **Nothing here demonstrates iPayMoney compatibility.** What it demonstrates is that *our* boundary — signature over raw bytes, reference lookup, amount equality, legal transition, idempotent application — exists and holds under adversarial input.

The order domain depends on the `PaymentProvider` interface that Phase 1 defined, never on the mock. `initiatePayment` and `applyProviderEvent` both take a provider as an argument; neither imports one. Replacing `MockPaymentProvider` with `IPayMoneyPaymentProvider` is a change at the composition root, not in the order domain.

Three changes to the mock, each with a reason

CHANGE	WHY
Charges are created pending , not succeeded	Answering "succeeded" synchronously permits an order domain written as <i>call the provider, believe the reply</i> , and that shape does not survive contact with a hosted-redirect provider. The asynchronous default forces the verification boundary to exist and be exercised by every test. The synchronous mode remains and is tested.
Permitted under a production build only with <code>ALLOW MOCK PAYMENTS=yes</code>	The browser suite runs <code>next start</code> , which means <code>NODE_ENV=production</code> . Exempting the test suite would have left the guard untested exactly where it matters. This is the two-variable shape <code>ConsoleSender</code> already uses: <code>PAYMENT_PROVIDER</code> is the variable a deployment gets wrong, and getting it wrong still cannot select the mock.
References carry an instance nonce	A per-process counter restarted at <code>1</code> and collided with rows a previous run had written, under the unique index Afrinext keeps on provider references. A real provider's reference identifies a charge for all time; the mock's now does too.

The outcome a mock charge reaches is decided by its *reference*, which is derived from the order — so a test chooses success or failure when it creates the product it is going to buy, the way a real provider's outcome is decided by the buyer's funds rather than by our code path.

Eleven deliberate defects, each applied to the real source, run against the suite, then reverted. Every mutation asserts its pattern was found before its result is believed — a mutation that does not apply proves nothing and reads exactly like a test gap.

DELIBERATE DEFECT	RESULT
Checkout ignores <code>status = 'published'</code> on the product and its store	CAUGHT
The webhook stops comparing the amount and currency	CAUGHT · 2 TESTS
The webhook accepts an unverified event instead of refusing it	CAUGHT · 3 TESTS
The payment <code>UPDATE</code> drops its <code>and status = ...</code> guard	CAUGHT
The order <code>UPDATE</code> drops its <code>and status = 'pending_payment'</code> guard	CAUGHT
The entitlement insert drops <code>on conflict do nothing</code>	CAUGHT
The transition table is widened so terminal states are no longer terminal	CAUGHT
<code>requireAccountConsent</code> is removed from checkout	CAUGHT
<code>authorize()</code> is removed from checkout	CAUGHT
Reading an order stops scoping to the buyer	CAUGHT
A ledger capture is posted on confirmation	CAUGHT

TWO WERE NOT CAUGHT AT FIRST, AND THAT IS THE USEFUL PART

The duplicate-grant guard. Removing `on conflict do nothing` from the entitlement insert changed nothing, because no test could reach it twice — the guarded transition already prevents a second attempt. So I looked for the case that *does* reach it: a buyer may open two orders for the same product, since the ownership check reads entitlements and neither order has granted one yet. Confirming both is reachable, and both confirmations must succeed — the buyer really did pay twice, and refusing the second would leave a captured charge with no order to explain it. There is now one grant and two paid orders, which is the correct answer and one nothing was asserting.

The consent gate at checkout. A `pending_consent` account resolves to no actor, so the gate is unreachable in the running system and removing it broke nothing. "Unreachable today" is a property of `resolveActor`, and a checkout relying on it would be one refactor away from selling to an unconsented account. The test now constructs the actor directly to reach the gate anyway.

The last row is the one I would draw your attention to: **adding** a ledger posting on confirmation is caught, so the boundary in section 1 is enforced by a test rather than by my intention to remember it.


```
pnpm lint                exit 0
pnpm typecheck            exit 0
pnpm test                 284 passed | 3 skipped (20 files)    was 234
pnpm build                exit 0
playwright test           22 passed                          was 18 · production mode, DB f
ledger concurrency        6 passed
reconcile-check           clean, zero drift
pnpm audit --audit-level high No known vulnerabilities found
mutation testing          11 of 11 caught
```

New: **46 domain tests** in `orders/orders.test.ts`, **4 browser tests** in `checkout.spec.ts`, and 5 added to the payments suite. **No new dependency**; `pnpm-lock.yaml` is untouched.

CI

CI, on this commit: run #18 (32383709772) on e9e16d5 — **ALL THREE JOBS GREEN**, 2 m 16 s. That includes a database built from zero, the schema rebuilt a second time to prove the migration applies to an empty cluster, the ledger concurrency suite, the reconciliation check, and the browser suite against a production build.

30 files

FILE	WHY
The domain	
core/orders/state.ts	New. Both state machines as data, plus the one place a payment state maps to an order state.
core/orders/checkout.ts	New. Pricing, visibility, idempotent order creation, buyer-scoped reads, entitlement listing.
core/orders/payment.ts	New. Initiation, the five-check verification boundary, the confirming transaction, expiry, frozen economics.
core/orders/orders.test.ts	New. 46 tests: transitions, pricing, idempotency, concurrency, adversarial webhooks, scoping, evidence.
core/orders/index.ts · core/index.ts	Exports.
Schema	
db/migrations/0009 + schema/commerce.ts	<code>orders</code> , <code>order_items</code> , <code>payments</code> , <code>payment_events</code> , <code>entitlements</code> . Status checks, the partial unique index for one live payment per order, the unique event id, and an append-only trigger on <code>payment_events</code> .
Payments	
core/payments/mock.ts	Asynchronous by default; unique references; an <code>event()</code> helper that produces genuinely signed — or deliberately forged — callbacks. Not a back door: what it returns still has to survive every check.
core/payments/registry.ts · payments.test.ts	The <code>ALLOW MOCK PAYMENTS</code> opt-in, and tests for both halves of it. See section 8 for the two existing assertions this changed.
Authorization	
core/authz/permissions.ts	<code>order.create</code> and <code>order.read_own</code> on <code>member</code> ; <code>order.read_store</code> on <code>store_owner</code> and <code>support</code> . In the seed, not a migration — the milestone-2 lesson.
Web	
web/api/v1/checkout/route.ts	Slugs and a key. 201 for a new order, 200 for a replay.
web/api/v1/orders/[id]/pay/route.ts	Empty body by design: no parameter can influence what is charged.
web/api/v1/payments/webhook/route.ts	Raw bytes, no session, one opaque refusal.
web/lib/payments.ts	The provider singleton, and the production guard.
web/lib/order-actions.ts · BuyButton · PayButton	Server Actions calling the same domain functions the API calls, so a form post and an API call cannot disagree.

FILE	WHY
web/[locale]/orders/ · product page	Order detail, order list, library; a real buy button, and an "already owned" state read from <code>entitlements</code> .
web/e2e/checkout.spec.ts	New. The buyer journey, the forged-confirmation attempt, price/replay, and scoping.
web/e2e/global-setup.ts · playwright.config.ts	Checkout tables cleared between local runs; the mock's two variables and its webhook secret passed to the server.
core/test/harness.ts	The new tables in the truncation list, ordered before the catalogue they reference.
i18n fr.json · en.json	19 keys per locale.
.env.example	<code>NODE_ENV</code> removed — section 8.

Risks, and the confirmations you asked for

CONFIRMATION	
No real iPayMoney integration was added	CONFIRMED
No production payout, withdrawal, refund or referral commission was added	CONFIRMED
No ledger or money-model change. <code>ledger/</code> and <code>money/</code> have no diff; money is <code>bigint</code> minor units throughout and no float appears in this milestone	CONFIRMED
OTP, consent, RBAC and catalogue guarantees intact — the OTP suite, the consent suites and the catalogue suite are unedited and pass	CONFIRMED
Nothing bypasses <code>pending_consent</code> ; <code>seller_terms</code> enforcement is untouched	CONFIRMED
No new dependency	CONFIRMED

Two existing tests changed, and why that is not a weakening

BOTH IN PAYMENTS.TEST.TS, BOTH REPORTED HERE RATHER THAN BURIED

"captures a charge and reports it" asserted `createCharge` returns `succeeded` . The mock is now asynchronous by default, so it asserts `pending` — and a new test covers the synchronous mode, so the old behaviour is still exercised rather than dropped.

"never hands out the mock in production" matched the old refusal message. It now matches the new one, still asserts both the registry and the constructor refuse, and additionally asserts that `ALLOW MOCK PAYMENTS` must be exactly `"yes"` . Strictly more coverage than before.

Open risks

ITEM	STATE
A payment can succeed against an order that has expired. The payment moves, the order does not, and nothing grants access — correct, but it means a buyer may have been charged for nothing. Nothing reconciles or refunds it yet, because refunds are not in scope. This is the highest-value thing to build next.	NEEDS WORK
Nothing sweeps expired checkouts on a schedule. <code>expireLapsedOrders()</code> exists and is tested; no job calls it. An order past its expiry is still refused at initiation, so the effect is stale rows rather than a wrong outcome.	UNSCHEDULED
Digital delivery does not exist. An entitlement is a row saying someone owns a product; there is no file, no download, no course access. The library screen links back to the product page.	NEXT MILESTONE
The commission rule must exist or a confirmation rolls back. Freezing the economics is inside the confirming transaction, so a missing rule for a product kind means the webhook 500s and the provider retries. That is the safe failure — but it is an outage until an operator adds the rule, and no alert fires.	WATCH
The mock keeps charges in memory, so a server restart forgets them and a later event for an older charge is refused as unknown. Harmless for a mock; noted because it would be a data-loss bug in a real adapter.	ACCEPTED
No refund path, no dispute path, no partial payment, no multi-item basket (the schema carries quantity and a line table, so a basket is a shape change rather than a new table).	OUT OF SCOPE

One thing found on the way

```
.ENV.EXAMPLE NO LONGER SETS NODE_ENV
```

Setting it there overrides every command that knows better: `next dev`, `next build`, `next start` and `vitest` each set the right value for themselves. With `NODE_ENV=development` exported from the file, `pnpm build` failed while prerendering the global error page — and it is the *same root cause* as last milestone's CI failure, where the browser suite served a development build while CI served a production one. One line removed; both symptoms gone.

Proposed next, in the order I would take them: digital delivery for a paid entitlement, then the reconciliation and refund path for the expired-order case above, then settlement — which is the first thing that should touch the ledger, and the first that needs a real provider. **Not started.**